

Computer Hardware

Memory and Cache

Ksawery Moźdzyski

Agenda For Today

1. The Memory Wall

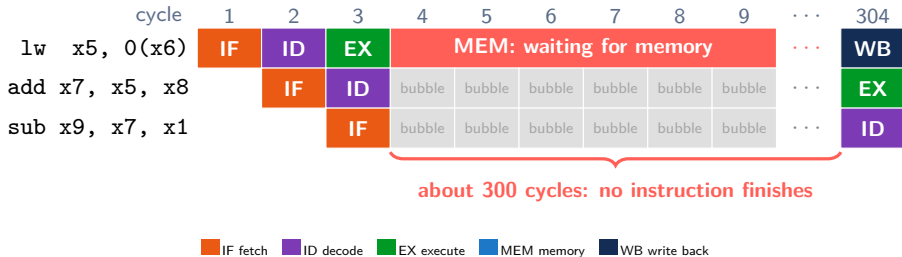
Why memory cannot be fast *and* cheap



2. Cache

Locality, hits and misses, mapping, AMAT

The problem: the pipeline waits for memory



The pipeline assumes that MEM takes **one** cycle. Main memory needs about **100 ns** $\approx$ **300 cycles** at 3 GHz: **300 instructions** that could have finished, and none did.

A processor is only as fast as the data it gets. Today: **where the data comes from.**

How far away is the data?

Imagine that one clock cycle (about 0.3 ns at 3 GHz) takes **one second**.

	Where	Real time (approx.)	If 1 cycle = 1 second
Orange square	Register	0.3 ns	1 second
Purple square	L1 cache	1 ns	a few seconds
Purple square	L2 cache	4 ns	about 13 seconds
Purple square	L3 cache	15 ns	about 1 minute
Blue square	Main memory (DRAM)	100 ns	about 5 minutes
Dark blue square	NVMe SSD	20–100 μ s	about 1–4 days
Dark blue square	Hard disk (HDD)	5–10 ms	about 6–12 months

From the CPU's point of view, RAM is a **coffee break** and a hard disk is a **year off**.

No memory is both fast and cheap

SRAM (used for caches)



6 transistors
per bit

- + answers in about 1 ns
- very expensive per bit: gigabytes would cost far too much

DRAM (used for main memory)



1 transistor
+ capacitor

- + cheap per bit: gigabytes are easy
- about 100 ns: the CPU waits hundreds of cycles

Flash (used in SSDs)

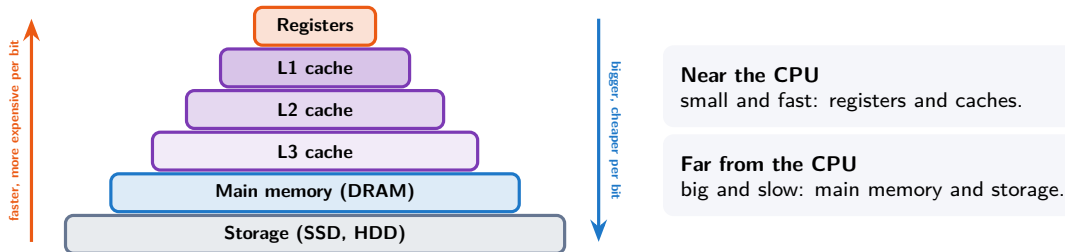


disk / flash

- + very cheap per bit, keeps data without power
- 20–100 μ s: far too slow for every access

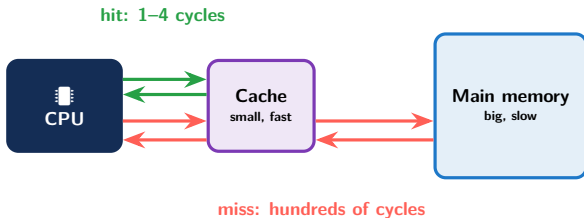
Fast memory is **expensive**, cheap memory is **slow**: no single technology gives us both.

The memory hierarchy



Goal: memory that **feels** as fast as the top and is as big as the bottom.

What is a cache? Hits and misses



Cache: a small, fast memory that keeps **copies** of recently used data from main memory.

Hit

The data is in the cache: fast.

Miss

The data is not there: fetch it from main memory, keep a copy, then use it.

The cache does not add new memory: it keeps **copies** close to the CPU. The program cannot see it.

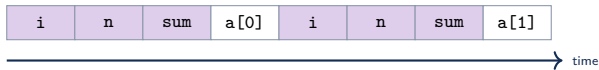
Why caching works: temporal locality

Temporal locality: data used now will probably be used again soon.

```
for (i = 0; i < n; i++)
    sum = sum + a[i];
```

sum, i and n are used in **every** iteration.

the same few addresses, again and again



What the cache does

It keeps **recently used** data close to the CPU, because it will probably be needed again.

Temporal locality: what was used **recently** is worth keeping close.

Task

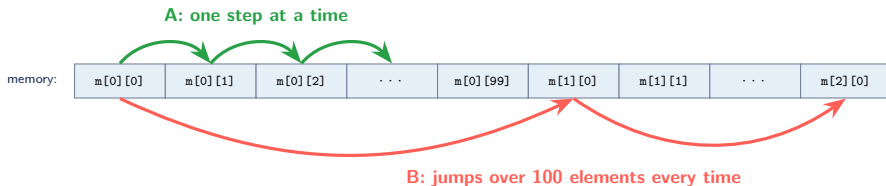
1. Which variables show **temporal** locality?
2. In memory, is `m[0][1]` next to `m[0][0]` or next to `m[1][0]`?
3. Which version has better **spatial** locality? Why?

```
for (i = 0; i < 100; i++)
    for (j = 0; j < 100; j++)
        total = total + m[i][j];
```

```
for (j = 0; j < 100; j++)
    for (i = 0; i < 100; i++)
        total = total + m[i][j];
```

memory:	m[0][0]	m[0][1]	m[0][2]	...	m[0][99]	m[1][0]	m[1][1]	...	m[2][0]
---------	---------	---------	---------	-----	----------	---------	---------	-----	---------

Solution: find the locality



1. Temporal

total, i and j are used in every iteration.

2. Layout

`m[0][1]` comes right after `m[0][0]`:
the array is stored row by row.

3. Spatial

Version **A** reads neighbours one after another. Version B jumps.

Same result and the same work, but version B **jumps around** in memory. Next: measure what that costs.

Exercise 2: measure both versions

Task

Put the two loops from Exercise 1 into a C program and measure how long each one takes.

How

Use a **big** array: `static int m[4000][4000];` (64 MB),
and fill it first.

Measure each loop with `clock()` from `<time.h>`.

Compile with optimisation: `gcc -O2`.

1. Which version is faster? **How many times** faster?
2. Why do we need a 4000×4000 array and not 100×100 ?

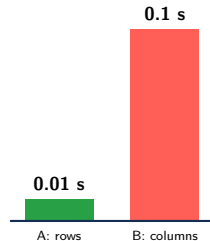
Solution: measure both versions

1. Result

On a test machine: A takes about **0.01 s**, B about **0.1 s**: B is about **10× slower**. Your numbers will differ, but B is clearly slower.

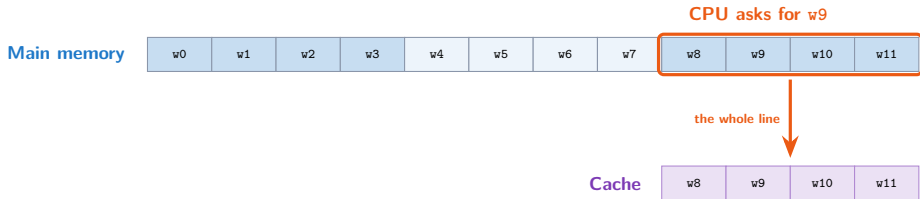
2. Why so big?

100×100 ints = 40 KB: the whole array fits in the cache, so both versions hit and the difference disappears. $4000 \times 4000 = 64$ MB does not fit.



Same work, same result: only the **order** of the memory accesses changed.

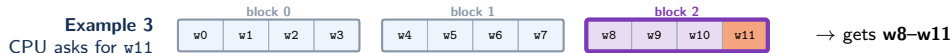
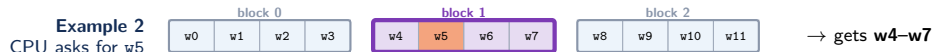
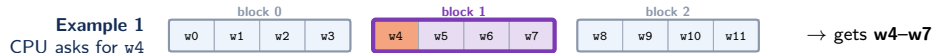
The cache copies whole lines



Cache line (block): the unit copied between main memory and the cache. In today's processors: **64 bytes** = 16 words of 4 bytes. In our pictures: **4 words**, to keep them small.

One miss brings the whole line, so the next accesses to its neighbours are **hits**.

Which words come along?

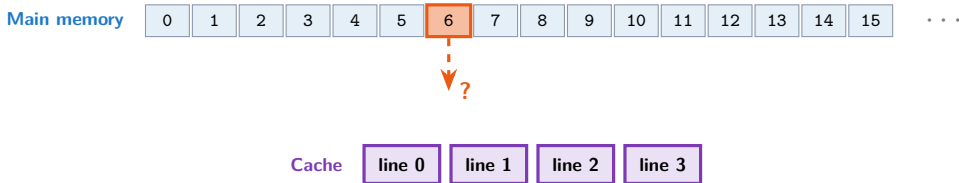


Fixed blocks

Memory is split into fixed blocks of 4 words, and every word belongs to exactly **one** block. The whole block is copied, so the neighbours can be on the **right**, on the **left** or on **both** sides.

The block boundaries **never move**: the requested word can be at the start, in the middle or at the end.

The cache is small

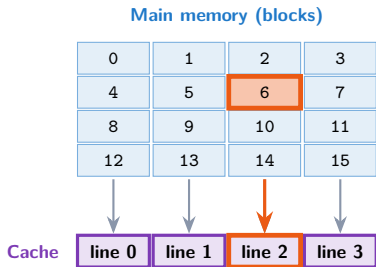


The problem

Main memory has millions of blocks, the cache only a few lines (here: 4). Block 6 has just been read: **which line** should keep it?

We need a simple, fast rule: **block number** → **line**.

Direct mapping: the column is the line



Draw memory with 4 columns

one column for each line of the cache. Every block goes to the line **under its column**.

The rule

line = block number **mod** 4

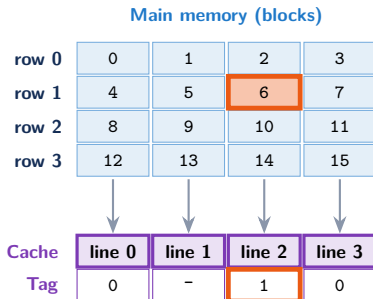
block 6 $\rightarrow 6 \bmod 4 = 2 \rightarrow$ line 2

Sharing

Blocks 2, 6, 10 and 14 all use line 2: only one of them can be in the cache at a time.

Direct-mapped cache: every block has exactly **one** line, so there is only one place to look.

How does the cache know which block it holds?



The problem

The CPU does not know whether line 2 holds block 2, 6, 10 or 14. The line must be **marked** somehow.

The tag

Next to the data, every line stores a tag: the **row** of its block.

$$\text{tag} = \left\lfloor \frac{\text{block number}}{4} \right\rfloor \quad \text{block 6: } \left\lfloor \frac{6}{4} \right\rfloor = 1$$

Empty lines

A **valid bit** marks whether a line holds a block at all (line 1: nothing yet).

Hit: the line of the block is valid and holds the **same tag**.

Exercise 3: line and tag

Task

A direct-mapped cache with **4 lines**, as in the pictures: $\text{line} = \text{block} \bmod 4$, $\text{tag} = \lfloor \text{block}/4 \rfloor$.

1. For each block, give its line and its tag:

Block	Line	Tag
5	...	...
12	...	...
7	...	...
9	...	...

2. The cache now holds:

Line	0	1	2	3
Tag	3	2	empty	1

Is each access a **hit** or a **miss**? Block **9**, block **13**, block **6**, block **7**.

 [ksawerym.github.io/ComputerHardwareSimulators/#cache](https://github.com/Ksawerym/ComputerHardwareSimulators/#cache)

Solution: line and tag

1. Line and tag

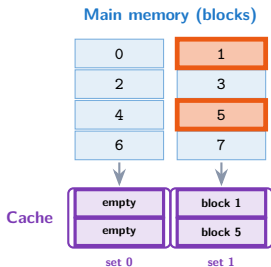
Block	Line (mod 4)	Tag ($\lfloor /4 \rfloor$)
5	1	1
12	0	3
7	3	1
9	1	2

2. Hit or miss?

Block	Line	Tag	Result
9	1	2	hit : line 1 holds tag 2
13	1	3	miss : line 1 holds tag 2 (block 9)
6	2	1	miss : line 2 is empty
7	3	1	hit : line 3 holds tag 1

Block 13 misses although line 1 is full: the line holds a **different** block with the same line number.

Set-associative caches: two places for every block



	1	5	1	5	1	5	
direct-mapped	M	M	M	M	M	M	0%
2 lines per set	M	M	H	H	H	H	67%

The problem

In a direct-mapped cache, blocks 1 and 5 both need line 1 and keep throwing each other out.

The fix

Group the lines into **sets**: $\text{set} = \text{block} \bmod 2$. A block may use any line of its set, so 1 and 5 fit together. When a set is full, the **least recently used** line goes out (LRU).

Real L1, L2 and L3 caches are **set-associative**, typically with 8 to 16 lines per set.

Exercise 4: hits and misses by hand

Task

A cache with **4 lines**, 1 block per line, initially empty. The program reads blocks **0, 4, 1, 0, 4, 1**. For every access write **H** (hit) or **M** (miss).

	0	4	1	0	4	1
direct-mapped	?	?	?	?	?	?
2 lines per set	?	?	?	?	?	?

1. **Direct-mapped**: $\text{line} = \text{block} \bmod 4$.
2. **2 lines per set**: the same 4 lines, 2 sets, $\text{set} = \text{block} \bmod 2$.
3. What is the **hit rate** of each?

 ksawerym.github.io/ComputerHardwareSimulators/#cache

Solution: hits and misses by hand

	0	4	1	0	4	1	
direct-mapped	M	M	M	M	M	H	17%
2 lines per set	M	M	M	H	H	H	50%

Direct-mapped: 0 and 4 both use line 0 and throw each other out. Block 1 has line 1 for itself, so its second access hits.

2 lines per set: 0 and 4 share set 0, which has two lines: after the first reads, everything hits.

Compulsory miss

The first access to a block always misses.

Conflict miss

Another block that uses the same line (or set) threw it out.

Capacity miss

The data simply does not fit in the cache.

More places per block removes **conflict** misses, but not compulsory or capacity misses.

How good is a cache? Hit rate, miss penalty, AMAT

Hit rate: the fraction of accesses that hit.

Miss rate = $1 - \text{hit rate}$.

Hit time: how long a hit takes.

Miss penalty: the extra time to bring the line from the next level.

Average Memory Access Time

$$\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}$$

Misses are rare, but each one costs many cycles: AMAT combines both.

Exercise 5: AMAT

Part A

An L1 cache with:

- hit time = 1 cycle
- miss rate = 4%, miss penalty = 10 cycles

1. What is the AMAT?
2. A program makes 1 000 000 memory accesses. How many cycles do they take? And with no misses at all?

Part B: which cache is better?

Two designs of an L1 cache:

- **Design A:** hit time 1 cycle, miss rate 8%, miss penalty 25 cycles
- **Design B:** hit time 2 cycles, miss rate 3%, miss penalty 25 cycles

Compute the AMAT of each. Which one is better, and why?

Part C (if you finish early): L1 hit time 1 cycle, miss rate 5%. L2 hit time 10 cycles, and 20% of the L1 misses also miss in L2. Main memory: 200 cycles. What is the AMAT with the L2 cache? And without it?

Solution: AMAT

Part A

$AMAT = 1 + 0.04 \times 10 = 1.4$
cycles

$1\,000\,000 \times 1.4 = 1\,400\,000$ cycles;
with no misses: 1 000 000

A miss rate of only 4% makes
memory access **40% slower**.

Part B

A: $1 + 0.08 \times 25 = 3.0$ **cycles**

B: $2 + 0.03 \times 25 = 2.75$ **cycles**

B is better, even though each of its
hits is twice as slow.

Part C

With L2:

$1 + 0.05 \times (10 + 0.2 \times 200) = 3.5$
cycles

Without L2:

$1 + 0.05 \times 200 = 11$ **cycles**

Misses are rare but **expensive**, so a lower miss rate often beats a faster hit.

Exercise 6: see the cache in Ripes

```
● ● ●
.data
arr: .zero 1024      # 256 words
.text
li    x9, 2          # two passes
pass: la    x5, arr   # start of arr
li    x6, 32         # N: reads per pass
loop: lw    x7, 0(x5)
      add   x10, x10, x7
      addi  x5, x5, 4  # STRIDE in bytes
      addi  x6, x6, -1
      bnez  x6, loop
      addi  x9, x9, -1
      bnez  x9, pass
```

Ripes task

Open the **Cache** tab. Data cache: direct-mapped, **16 lines**, **4 words per line** (256 bytes). **Predict first**, then run and read the hit rate.

Run	STRIDE	N	Predicted	Ripes
1	4	32	...	...
2	16	32	...	...
3	16	16	...	...
4	4	128	...	...

Which run shows a **capacity** problem? Which one shows pure **spatial locality**?

Solution: see the cache in Ripes

Run	Reads	Misses	Hit rate	Why
1: stride 4, N 32	64	8	87.5%	1 miss per 4 words; 128 B fit, so pass 2 only hits
2: stride 16, N 32	64	64	0%	every read is a new line; 32 lines > 16 lines, so pass 2 misses too
3: stride 16, N 16	32	16	50%	pass 1 always misses, but 16 lines fit, so pass 2 hits
4: stride 4, N 128	256	64	75%	512 B > 256 B: nothing is left from pass 1

Runs 1 and 2

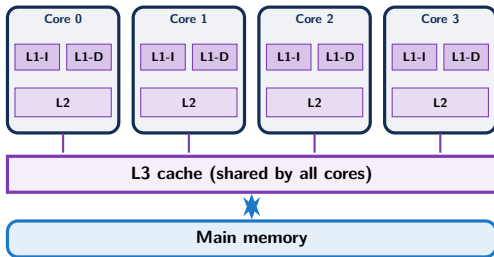
The same number of reads, but spatial locality is lost: the hit rate falls from 87.5% to 0%.

Runs 1 and 4

The same stride, but the data stopped fitting in the cache: **capacity** misses. Run 4 shows **pure spatial locality**: nothing is reused, every hit is a neighbour in the same line.

The same instructions with a different memory pattern: the hit rate can go from **87.5% to 0%**.

Many levels of cache



L1

tens of KB, about 4 cycles, **split** into instructions (I) and data (D)

L2

hundreds of KB to a few MB, about 12 cycles, one per core

L3

tens of MB, about 40 cycles, **shared** by all cores. The same SRAM as L1: **bigger is slower**.

Split L1 = separate paths for instructions and data = the **modified Harvard** architecture. It also removes the structural hazard between IF and MEM in the pipeline.

Each level catches most misses of the level above, so only **few** accesses go all the way to main memory.

Real-world reveal: two laptops, one idea

	Intel Core i9-14900HX	Apple M3
Instruction set	x86-64 (CISC outside)	ARM64 (RISC)
Cores	8 performance + 16 efficiency	4 performance + 4 efficiency
L1 per performance core	32 KB (I) + 48 KB (D)	192 KB (I) + 128 KB (D)
L2	2 MB per performance core	16 MB shared by the performance cores
Last level	36 MB L3, shared by all cores	system level cache, shared
Main memory	separate DDR5 modules	unified memory in the chip package

Two different instruction sets, the **same** idea: a small split L1, a bigger L2, a big shared last level.

Task Manager → Performance → CPU
(L1, L2, L3 at the bottom right)



```
lscpu | grep -i cache
```



```
sysctl -a | grep cachesize
```

Write down the sizes of your L1, L2 and L3 caches. Is your L1 split into instructions and data?

Software can **read** the cache sizes, but it cannot choose what is inside: the hardware decides.

Solution: what cache does your laptop have?

Example: Intel Core i9-14900HX, Linux

```
● ● ●  
$ lscpu | grep -i cache  
L1d cache: 896 KiB (24 instances)  
L1i cache: 1.3 MiB (24 instances)  
L2 cache: 32 MiB (12 instances)  
L3 cache: 36 MiB (1 instance)
```

Split L1?

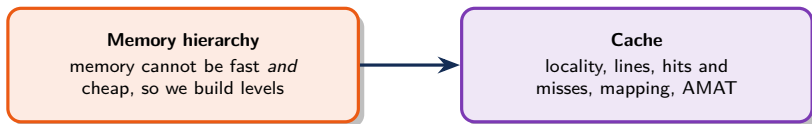
Yes: **L1d** (data) and **L1i** (instructions) are separate caches.

Totals

The numbers add up all cores. L3 has **1 instance**: one L3 shared by all cores.

The same structure everywhere: a split L1, a bigger L2 and one big **shared** L3.

What we saw today



A fast CPU is only as fast as the data it gets: the cache keeps that data **close**.

